EagleEye

Cloud (Remote) Firmware

Technical Reference & Setup Guide

*The remote / product firmware for the ESP32–CAM surveillance node: camera at a site, phone anywhere. The device connects **outbound** to the cloud and stays connected, so its IP never matters.*

Module	`firmware/eagleeye-cloud/`
Target	ESP32–CAM (AI Thinker, PSRAM)
Firmware	v1.0.0-cloud
Plan	`distributed-roaming-candy.md`
Status	Phases P1–P4 implemented (device side)

Independent of `firmware/eagleeye-main/` (the LAN build is untouched).

Contents

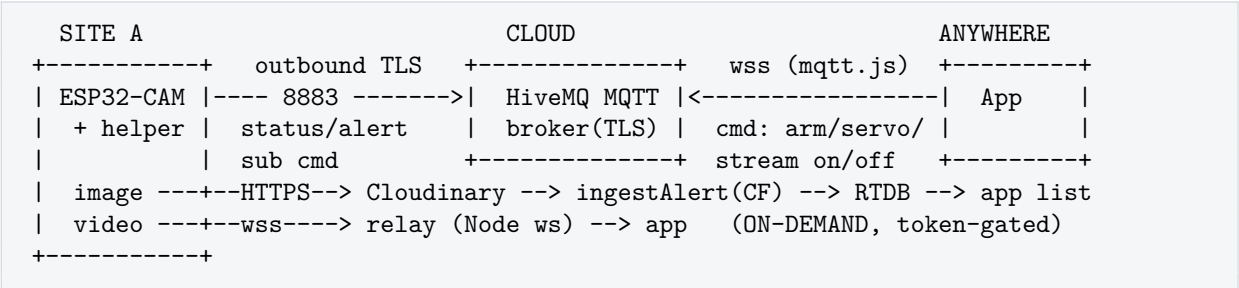
1 Overview

This document describes the **cloud / remote** build of the EagleEye ESP32–CAM firmware. Unlike the LAN build in `firmware/eagleeye-main/` (where the phone reaches *into* the camera by its local IP), this build inverts the topology: the device dials **out** to cloud services and holds the connection open — exactly how a phone receives push messages anywhere without a fixed IP. This removes the manual-IP problem, the port-forwarding/NAT problem, and the dependency on a PC at the site.

It implements the approved plan `distributed-roaming-candy.md`:

- **Plane 1 — Control & alerts** over a managed cloud MQTT broker (TLS).
- **Plane 2 — Live video** via an on-demand cloud WebSocket relay.
- **Direct HTTPS upload** of intruder images to Cloudinary (no PC bridge).
- **Phase 4** — Wi-Fi provisioning portal, HTTPS OTA, and TLS hardening.

2 Architecture



Two planes with different needs: **control + alerts** are small, reliable, and always-connected (MQTT/TLS + HTTPS + RTDB); **video** is bursty and high-bandwidth (on-demand relay, hardware-JPEG frames).

3 File Map

File	Role
`eagleeye-cloud.ino`	Main sketch: AI loop, `DeviceMode` dispatch, command handling, setup.
`config.h`	All settings (fill the `DEV_*` defaults) + NVS load/save + topic helpers + feature flags.
`eagleeye_camera.h`	Camera pins; `MODE_AI` (RGB565) vs `MODE_RELAY` (hardware JPEG); safe mode switch; `eagleeye_grab_jpeg`.
`EagleEye_Cloud_IoT.h`	MQTT-over-TLS: retained status + LWT, `cmd` parsing, alert publish, SNTP, non-blocking reconnect, `capture_and_send_image`.
`eagleeye_upload.h`	Direct HTTPS upload to Cloudinary (unsigned preset) + `ingest_alert` to a Cloud Function.
`eagleeye_relay.h`	Outbound `wss` relay client (Plane 2); on-demand with idle / max-session caps.
`eagleeye_ota.h`	MQTT-triggered HTTPS OTA (Phase 4).
`eagleeye_provision.h`	Wi-Fi captive-portal setup (Phase 4, off by default).

4 Required Arduino Libraries

- **PubSubClient** (knolleary) — MQTT.
- **ArduinoJson** (v6) — JSON encode/decode.
- **WebSockets** (Markus Sattler / Links2004) — relay client.
- **final_inferencing** — the EI v7.16 RGB 96 × 96 library (same one `eagleeye-main` uses).
- **WiFiManager** (tzapu) — *only if* `ENABLE_PROVISIONING` is set to `1`.

Board: **AI Thinker ESP32–CAM**, PSRAM enabled. For OTA, choose an **OTA-capable Partition Scheme** (Arduino IDE → Tools → Partition Scheme).

5 Configuration (`config.h`)

Edit the `DEV_*` defaults before flashing: Wi-Fi credentials, HiveMQ host / user / password, Cloudinary cloud name and **unsigned upload preset**, and (later) the Cloud Function URLs and relay host. These are compile-time defaults; the Phase-4 portal can override them at runtime into NVS, so no re-flashing is needed to change networks in the field.

6 Cloud Setup (Your Accounts)

1. **HiveMQ Cloud** (free tier): create a cluster and two credentials with topic permissions — a *device* credential (publish `status/alert/stream`, subscribe `cmd`) and an *app* credential (publish `cmd`, subscribe the rest), each scoped to `eagleeye/<id>/#`.
2. **Cloudinary**: create an **unsigned upload preset** that forces `folder=eagleeye_intrusions`, `resource_type=image`, and a max size. The API secret is *never* placed on the device.
3. **Firebase Cloud Functions** (Phases 2–3): `ingestAlert` (writes RTDB `alerts/` so the app's existing list shows it), `onDeletionRequest` (Cloudinary destroy — replaces `bridge.py`), `issueStreamToken` (short-lived relay tokens).
4. **Relay** (Phase 3): a small Node `ws` server on Fly.io / Railway exposing `wss://`.

7 Topics & Command Format

```
eagleeye/<id>/status   retained + LWT   {"online":true,"armed":true,"fw":"...","rssi
    ":-62}
eagleeye/<id>/cmd      subscribe        {"type":"arm","value":true}
                                        {"type":"servo","angle":90}
                                        {"type":"stream","value":true}
                                        {"type":"ota","url":"https://.../fw.bin"}
                                        {"type":"factory_reset"}
eagleeye/<id>/alert    publish          {"ts":...,"image_url":...,"public_id":...,
                                         "score":...,"type":"Human Detected"}
eagleeye/<id>/stream   publish          {"ready":true}   (tells the app to attach to
    the relay)
```

8 TLS Memory (Design Rationale)

The ESP32–S1 cannot hold three TLS sessions at once. A single state machine `DeviceMode { MODE_AI, MODE_UPLOADING, MODE_RELAY }` (in `eagleeye_camera.h`) guarantees that **at most one of {HTTPS upload, WSS relay} is open at a time**. The persistent **MQTTS** session stays small because the old ∼50 KB image buffer is gone (`setBufferSize(1024)`); JPEGs now travel over HTTPS instead of MQTT. HTTPS upload clients are scope-local and freed immediately after each upload, so the heap is reclaimed at once.

9 Phase Status (This Firmware)

Phase	State	Notes
P1 control / status	Done	MQTTS, LWT, retained status, `cmd`→servo/arm/stream.
P2 upload + alert	Done	Cloudinary unsigned upload; `ingest_alert` no-ops until `DEV_INGEST_URL` is set.
P3 relay video	Done	Hardware-JPEG switch + `wss` client; needs relay host + `issueStreamToken`.
P4 prov / OTA / sec	Done*	OTA on by default; provisioning behind `ENABLE_PROVISIONING`; TLS uses `setInsecure()` for dev — set `TLS_INSECURE 0` and pin CA certs for production.

10 Bring-up Order (Recommended)

1. Fill Wi-Fi + HiveMQ credentials, flash, and watch serial for: Wi-Fi join, SNTP, `[MQTT] ok`, and a retained `status` message in the HiveMQ web client. Cut power → `online:false` appears via the LWT.
2. From the HiveMQ web client, publish to `eagleeye/cam-01/cmd`: `{"type":"servo","angle":120}` (servo moves) and `{"type":"arm","value":false}`.
3. Add the Cloudinary preset, trigger a detection → the image appears in `eagleeye_intrusions`. Deploy `ingestAlert` so it lands in RTDB and the app shows it; then retire Mosquitto + `bridge.py`.
4. Deploy the relay + `issueStreamToken`, set `DEV_RELAY_HOST` / `DEV_TOKEN_URL`, then `{"type":"stream","va` → frames flow; `false` stops.
5. Production: rotate the leaked Cloudinary / Firebase credentials, set `TLS_INSECURE 0` and pin CAs, and enable provisioning.

11 Security Note

The Cloudinary API secret in `backend/bridge.py` / `.env` and the committed Firebase `serviceAccountKey.jso` are **burned** — rotate them and purge them from git history before shipping. This firmware deliberately holds **no** Cloudinary secret (it uses an unsigned preset) and only its own scoped MQTT credentials.

EagleEye FYP — generated technical reference for the cloud firmware module. See `distributed-roaming-candy.md` for the full multi-phase plan including the mobile-app and cloud-service deliverables.